



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий (ИИТ)

Кафедра прикладной математики (ПМ)

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ №4

по дисциплине «Технологии и инструментарий машинного обучения»

Студент группы *ИНБО-20-23. Школьный И.В.*

(подпись)

Преподаватель *Трушин С.М.*

(подпись)

Москва 2025 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	3
1 МЕТОД РЕШЕНИЯ	4
ВЫВОД.....	5

ВВЕДЕНИЕ

Цель практической работы №4 — изучить методы классификации на примере логистической регрессии, освоить процесс построения и оценки моделей бинарной и многоклассовой классификации.

1 МЕТОД РЕШЕНИЯ

Необходимо подобрать датасет, в котором не менее 1000 строк, хотя бы 3 числовых признака и два категориальных. Импорт представлен в Листинге 1.

Листинг 1 — Импорт библиотек и датасета

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

df = pd.read_csv('loan_data.csv')
print(f'Размер датасета: {df.shape}')
df.head()
```

Так выглядят первые 5 строк датасета (Таблица 1).

Таблица 1 — Первые 5 строк датасета

person_age	person_gender	person_education	person_income	person_employment	person_home_ownership	loan_amount	loan_intent	loan_int_rate	loan_percent_income	cb_person_cred_hist_length	credit_score	previous_loan_defaults_on_file	loan_status
22.0	female	Master	71948.0	0	RENT	35000.0	PERSONAL	16.02	0.49	3.0	561	No	1
21.0	female	High School	12282.0	0	OWN	1000.0	EDUCATION	11.14	0.08	2.0	504	Yes	0
25.0	female	High School	12438.0	3	MORTGAGE	5500.0	MEDICAL	12.87	0.44	3.0	635	No	1
23.0	female	Bachelor	79753.0	0	RENT	35000.0	MEDICAL	15.23	0.44	2.0	675	No	1
24.0	male	Master	66135.0	1	RENT	35000.0	MEDICAL	14.27	0.53	4.0	586	No	1

Выполним масштабирование числовых признаков. Это процесс

преобразования числовых данных так, чтобы они находились в общем сопоставимом масштабе. Для этого воспользуемся функцией стандартизации `StandardScaler`, который хорош для большинства случаев. Код программы представлен в Листинге 2.

Листинг 2 — Масштабирование числовых признаков

```
numeric_cols = ['person_age', 'person_income', 'person_emp_exp', 'loan_amnt',  
'loan_int_rate', 'loan_percent_income', 'cb_person_cred_hist_length',  
'credit_score']  
scaler = StandardScaler()  
df[numeric_cols] = scaler.fit_transform(df[numeric_cols])
```

Далее выполним кодирование категориальных признаков так же, как и в прошлой работе (Листинг 3).

Листинг 3 — Кодирование категориальных признаков

```
categorical_cols = ['person_gender', 'person_education',  
'person_home_ownership', 'loan_intent', 'previous_loan_defaults_on_file']  
label_encoders = {}  
for col in categorical_cols:  
    le = LabelEncoder()  
    df[col] = le.fit_transform(df[col])  
    label_encoders[col] = le
```

После этого экспортируем файл, чтобы провести параллельную работу на `loginom`.

1.1 Задание 1. Логистическая регрессия

Начнем с реализации на Python. После того, как данные обработаны, необходимо разделить выборку на обучающую и тестовую. Код программы представлен в Листинге 4.

Листинг 4 — Разделение выборки

```
X = df.drop(columns=['loan_status'])  
y = df['loan_status']  
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,  
                                                    random_state=42, stratify=y)
```

Построим логистическую регрессию разными способами: простую, с

использованием регуляризации. Также выполним настройку параметра C (Листинг 5).

Листинг 5 — Построение логистической регрессии

```
# 1. Обычная логистическая регрессия
logreg = LogisticRegression(random_state=42, max_iter=1000)
logreg.fit(X_train, y_train)
y_pred = logreg.predict(X_test)
logreg_l1 = LogisticRegression(penalty='l1', solver='liblinear',
random_state=42, max_iter=1000)
logreg_l2 = LogisticRegression(penalty='l2', random_state=42, max_iter=1000)
param_grid = {'C': [0.001, 0.01, 0.1, 1, 10, 100]}
grid_l1 = GridSearchCV(logreg_l1, param_grid, cv=5, scoring='f1')
grid_l2 = GridSearchCV(logreg_l2, param_grid, cv=5, scoring='f1')
grid_l1.fit(X_train, y_train)
grid_l2.fit(X_train, y_train)
models = {
    'LogReg': logreg,
    'LogReg L1': grid_l1.best_estimator_,
    'LogReg L2': grid_l2.best_estimator_
}
for name, model in models.items():
    y_pred = model.predict(X_test)
    print(f"{name}:")
    print(f" Accuracy: {accuracy_score(y_test, y_pred):.2f}")
    print(f" Precision: {precision_score(y_test, y_pred):.2f}")
    print(f" Recall: {recall_score(y_test, y_pred):.2f}")
    print(f" F1: {f1_score(y_test, y_pred):.2f}")
    print(" Confusion Matrix:")
    print(confusion_matrix(y_test, y_pred))
    print()
```

Получаем следующий результат (Рисунок 1).

```
LogReg:
Accuracy: 0.90
Precision: 0.78
Recall: 0.75
F1: 0.76
Confusion Matrix:
[[6573 427]
 [ 503 1497]]

LogReg L1:
Accuracy: 0.90
Precision: 0.78
Recall: 0.75
F1: 0.77
Confusion Matrix:
[[6575 425]
 [ 496 1504]]

LogReg L2:
Accuracy: 0.90
Precision: 0.78
Recall: 0.75
F1: 0.76
Confusion Matrix:
[[6571 429]
 [ 501 1499]]
```

Рисунок 1 — Логистическая регрессия на Python

Проанализируем матрицы ошибок:

1. Простая логистическая регрессия: $TP = 1497$, $FP = 427$, $FN = 503$, $TN = 6573$.
2. L1-регуляризация: $TP = 1504$, $FP = 425$, $FN = 496$, $TN = 6575$.
3. L2-регуляризация: $TP = 1499$, $FP = 429$, $FN = 501$, $TN = 6571$.

Все три модели демонстрируют схожую эффективность, что говорит о том, что выборка не нуждается в регуляризации в силу отсутствия шумов.

Так выглядит график зависимости метрик от порога (Рисунок 2).

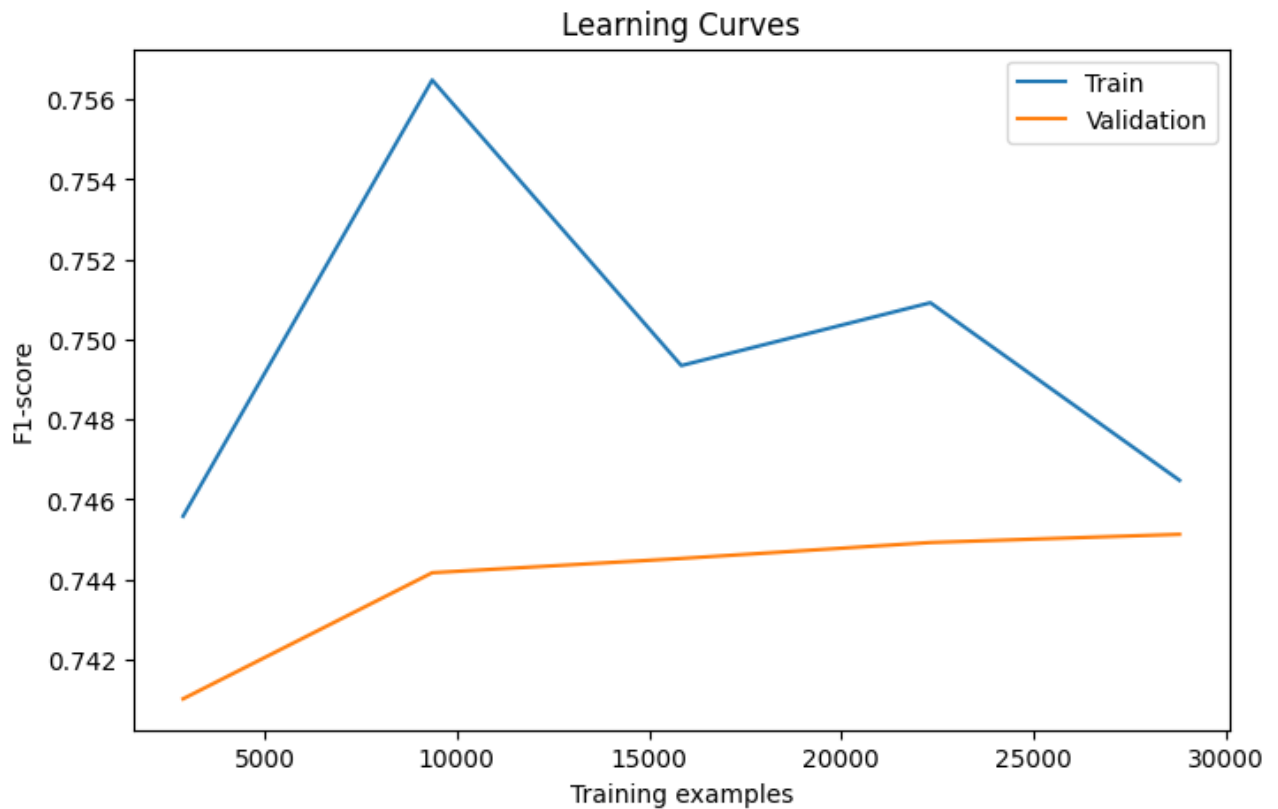


Рисунок 2 — График зависимости метрик от порога

Теперь попробуем сделать то же самое в `logitnet`. Добавляем необходимые узлы и настраиваем их (Рисунок 3).

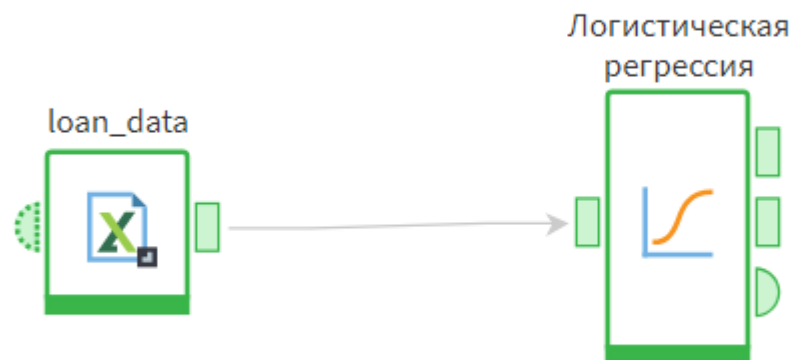


Рисунок 3 — Сценарий в loginom

Узел логистической регрессии предусматривает в своей настройке деление выборки на тестовую и обучающую. Посмотрим на сводку результатов (Рисунок 4).

Выход регрессии		Коэффициенты регрессионной модели		Сводка
№	Имя	Метка	Значение	
2	12 TotalSelectedSamples	Всего отобранных примеров	45 000	
3	12 TrainSamples	Примеров в обучающем множестве	36 000	
4	90 TrainRMSError	Среднеквадратическая ошибка на обучающем множестве	0,27	
5	90 TrainClsErrorPercentage	Процент ошибок классификации на обучающем множестве	10,94	
6	90 TrainAvgCE	Средняя перекрестная энтропия на обучающем множестве	0,33	
7	90 TrainThreshold	Порог отсечения при обучении модели	0,50	
8	90 NullMinusTwoLogL	-2 Логарифма функции правдоподобия нулевой модели	37 914,58	
9	90 MinusTwoLogL	-2 Логарифма функции правдоподобия	16 656,28	
10	90 R2	Псевдо-R ² Макфаддена	0,56	
11	90 AdjustedR2	Псевдо-R ² Макфаддена (скорр.)	0,56	
12	12 DFE	Число степеней свободы ошибки	35 986	
13	12 ModelDF	Число степеней свободы модели	13	
14	90 Chi2	Хи-квадрат	21 258,30	
15	90 AIC	Информационный критерий Акаике	0,46	
16	90 AICc	Информационный критерий Акаике скорректированный	0,46	
17	90 BIC	Информационный критерий Байеса	0,47	
18	90 HQC	Информационный критерий Ханнана-Куинна	0,46	
19	90 ModelPValue	P-значение модели	0,00	
20	12 TestSamples	Примеров в тестовом множестве	9 000	
21	90 TestRMSError	Среднеквадратическая ошибка на тестовом множестве	0,27	
22	90 TestClsErrorPercentage	Процент ошибок классификации на тестовом множестве	10,81	
23	90 TestAvgCE	Средняя перекрестная энтропия на тестовом множестве	0,33	
24	12 ValidationSamples	Примеров в валидационном множестве	7 200	
25	12 ValidationPartitions	Число валидационных разбиений	5	
26	90 CVRMSError	Среднеквадратическая ошибка при кросс-валидации	0,27	
27	90 CVClsErrorPercentage	Процент ошибок классификации при кросс-валидации	10,94	
28	90 CVAvgCE	Средняя перекрестная энтропия при кросс-валидации	0,33	

Рисунок 4 — Сводка

Точность модели составляет $100 - 10.81 = 89.19\%$, что говорит о том, что модель хорошо работает. Разница между среднеквадратичными ошибками выборок незначительна, что говорит об отсутствии переобучения.

Таким образом, были построены модели логистической регрессии с помощью двух разных инструментов, которые по-своему хороши.

1.2 Классификация при несбалансированности классов

Убедимся в том, что наши классы не сбалансированы (Листинг 6).

Листинг 6 — Несбалансированность классов

```
print(df['loan_status'].value_counts(normalize=True))
```

Вывод представлен на Рисунке 5.

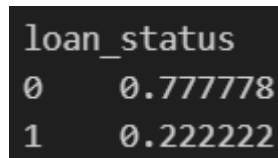


Рисунок 5 — Результат

Замечаем, что значений 0 гораздо больше, чем значений 1. Это понизит точность нашей модели, в данном случае, будет проблематичнее выдавать кредиты людям.

Выполним три разных варианта построения моделей: без балансировки, с балансировкой весов классов и методом oversampling. Наша задача повысить параметр recall (долю реально положительных случаев, которые модель правильно нашла). Построим ROC-кривые для сравнения результатов (Листинг 7).

Листинг 7 — Балансировка и анализ

```
logreg_unbalanced = LogisticRegression(random_state=42, max_iter=1000)
logreg_unbalanced.fit(X_train, y_train)

logreg_balanced = LogisticRegression(class_weight='balanced', random_state=42,
max_iter=1000)
logreg_balanced.fit(X_train, y_train)

smote = SMOTE(random_state=42)
X_train_smote, y_train_smote = smote.fit_resample(X_train, y_train)
logreg_smote = LogisticRegression(random_state=42, max_iter=1000)
logreg_smote.fit(X_train_smote, y_train_smote)

fig, ax = plt.subplots(figsize=(8,6))
for name, model in [('Unbalanced', logreg_unbalanced), ('Balanced Weights',
logreg_balanced), ('SMOTE', logreg_smote)]:
    y_prob = model.predict_proba(X_test)[:,-1]
    fpr, tpr, _ = roc_curve(y_test, y_prob)
    auc_score = auc(fpr, tpr)
    ax.plot(fpr, tpr, label=f'{name} (AUC = {auc_score:.2f})')
```

Продолжение Листинга 7

```
ax.plot([0,1], [0,1], 'k--')
ax.set_xlabel('FPR')
ax.set_ylabel('TPR')
ax.legend()
ax.set_title('ROC-кривые для разных методов балансировки')
plt.show()
```

Посмотрим на кривые (Рисунок 6).

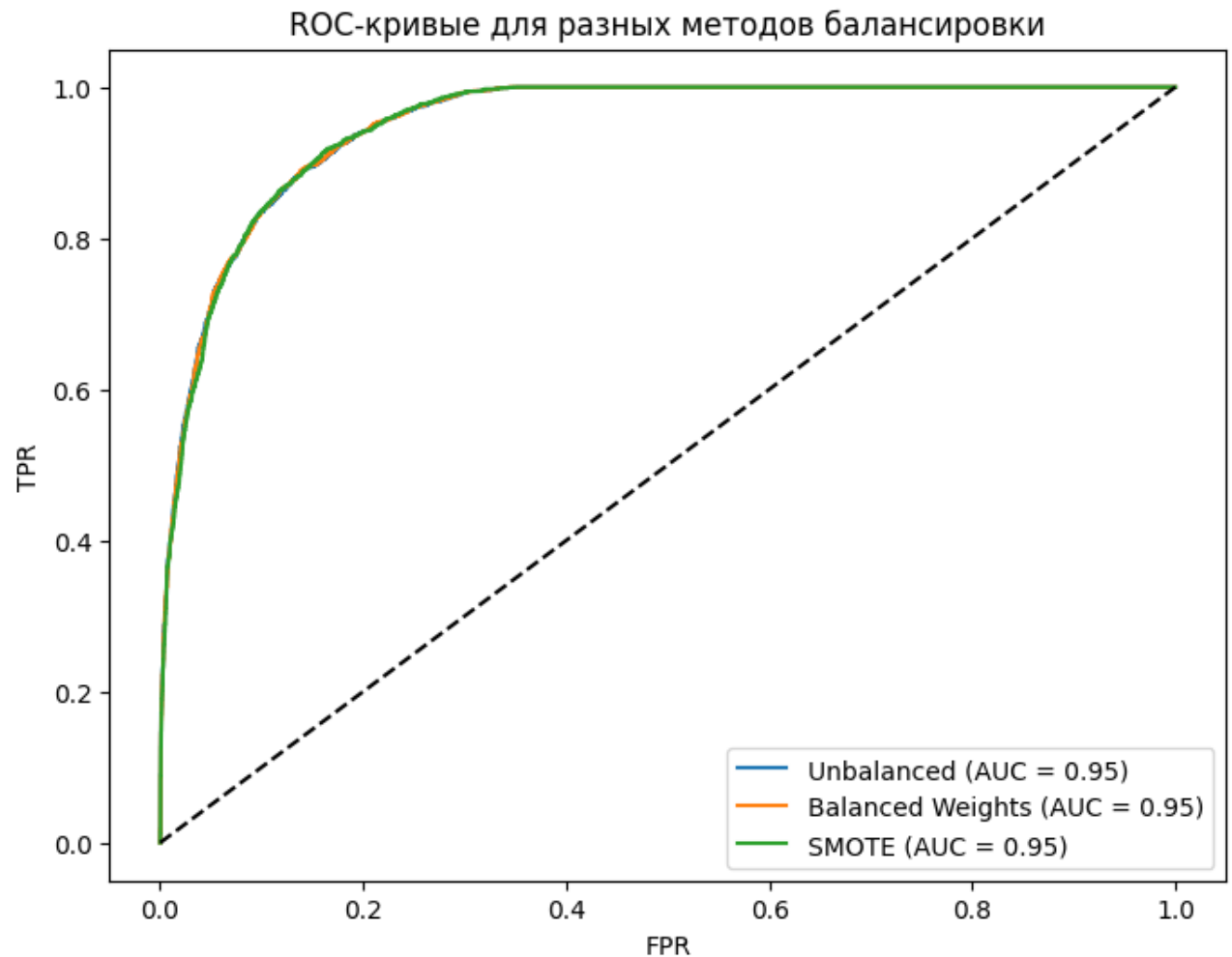


Рисунок 6 — ROC-кривые

Можем заметить, что все модели предсказывают с одинаковой точностью, значит балансировка сильно не влияет на результат.

1.3 Многоклассовая логистическая регрессия

Для многоклассовой классификации выберем другой признак за исследуемый — цель выдачи кредита. Построим модели и оценим результаты (Листинг 8).

Листинг 8 — Многоклассовая логистическая регрессия

```
X_multi = df.drop(columns=['loan_intent'])
y_multi = df['loan_intent']
X_train_m, X_test_m, y_train_m, y_test_m = train_test_split(X_multi, y_multi,
test_size=0.2, random_state=42, stratify=y_multi)
models_multi = {
    'LogReg (OvR)': LogisticRegression(multi_class='ovr', max_iter=1000),
    'kNN': KNeighborsClassifier(),
    'SVM (OvO)': SVC(decision_function_shape='ovo')
}
results = {}
for name, model in models_multi.items():
    model.fit(X_train_m, y_train_m)
    y_pred_m = model.predict(X_test_m)
    results[name] = {
        'Accuracy': accuracy_score(y_test_m, y_pred_m),
        'F1 (macro)': f1_score(y_test_m, y_pred_m, average='macro')
    }
results_df = pd.DataFrame(results).T
print(results_df)
```

В этом пункте делаем все то же разбиение выборки, составление моделей для сравнения и оценку просмотр метрик (Рисунок 7).

	Accuracy	F1 (macro)
LogReg (OvR)	0.236556	0.168229
kNN	0.195111	0.185411
SVM (OvO)	0.248778	0.179610

Рисунок 7 — Готовая таблица метрик

Точность достаточно низкая, это говорит о сложности предсказания цели взятия кредита по искомым признакам.

ВЫВОД

В ходе выполнения практической работы №4 были успешно изучены методы классификации на примере логистической регрессии, освоены процессы построения и оценки моделей бинарной и многоклассовой классификации.